

Learning with Gradient Descent

Plus Momentum, SGD, MiniBatch SGD and Adagrad/Adam

Reference Book Chapters: Chapter 7

The (Batch) Gradient Descent Alg.

Update the weights by moving in the **opposite direction** of the gradient:

```
 $\mathbf{w}^{(0)} = \text{Initialize}()$   
for  $\tau$  in range(1,  $\tau_{\text{max}}$ ):  
     $\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta \nabla J(\mathbf{w}^{(\tau-1)})$   
    if  $\text{Converged}(\mathbf{w}^{(\tau)}, \mathbf{w}^{(\tau-1)}, \epsilon)$ : terminate
```

Note: We write loss as $J(\mathbf{w})$ not $E(\mathbf{w})$ in this lecture

$\text{Initialize}()$ typically a small random value (more on this later)

$\text{Converged}(\mathbf{w}^{(\tau)}, \mathbf{w}^{(\tau-1)}, \epsilon)$ stopping condition (e.g., ϵ change in $\Delta \mathbf{w}^{(\tau-1)}$)

Hyper Parameters:

- η is the learning rate (typically a small value $0 < \eta < 1$, more on this later)
- τ_{max} is the maximum number of iterations

let's work on an example

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla J(\mathbf{w}^{(t)})$$

Let's start with a linear model:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{w} \text{ in } \mathbb{R}^2, b \text{ in } \mathbb{R}.$$

The unknown parameters are $\mathbf{w} = \{w_1, w_2, b\}$

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Training=Learning= Fitting = Optimizing parameters $\mathbf{w}$.

Write down the loss $J(\mathbf{w})$

Do gradient descent or SGD or Mini-Batch SGD on $J(\mathbf{w})$ to find the best parameters $\mathbf{w}^*$.

Goal: Find what that minimizes $J(\mathbf{w})$.

Loss= $J(\boldsymbol{\theta})$ = squared loss...

let's work on an example

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Let's start with a linear model:

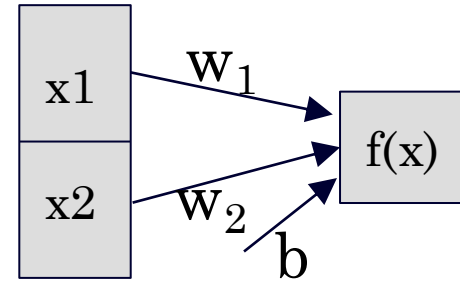
$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{w} \text{ in } \mathbb{R}^2, b \text{ in } \mathbb{R}.$$

The unknown parameters are $\theta = \{w_1, w_2, b\}$

$$\text{Loss} = J(\theta) = \frac{1}{4} \sum_{i=1}^n (f^*(\mathbf{x}_i) - f(\mathbf{x}_i))^2$$

let's work on an example

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0



$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$, $\mathbf{w}$ in $\mathbb{R}^2$, b in $\mathbb{R}$.
weights $\boldsymbol{\theta} = \{w_1, w_2, b\}$

Loss $J(\theta) = \frac{1}{4} \sum_{i=1}^n (f^*(\mathbf{x}_i) - f(\mathbf{x}_i))^2$

Exercise: Compute

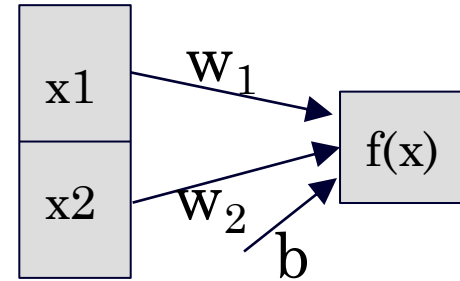
1. predictions $f(\mathbf{x})$ and
2. loss for $J(\boldsymbol{\theta})$

For $\boldsymbol{\theta} = \{w_1 = \mathbf{0}, w_2 = \mathbf{1}, b = \mathbf{1}\}$

using the data set shown in the table

let's work on an example

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0



$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$, $\mathbf{w}$ in $\mathbb{R}^2$, b in $\mathbb{R}$.
weights $\boldsymbol{\theta} = \{w_1, w_2, b\}$

$$\text{Loss} = J(\theta) = \frac{1}{4} \sum_{i=1}^n (f^*(\mathbf{x}_i) - f(\mathbf{x}_i))^2$$

Answer:

$$f(\mathbf{x}) = \begin{bmatrix} 1 \\ 2 \\ 1 \\ 2 \end{bmatrix}$$

$$\begin{aligned} \text{Loss } J(\boldsymbol{\theta}) &= \frac{1}{4} * ((0-1)^2 + (1-2)^2 \\ &\quad + (1-1)^2 + (0-2)^2) \\ &= \frac{1}{4} * (1+1+ 0+4) = \mathbf{6/4} \end{aligned}$$

Exercise: Compute

1. predictions $f(\mathbf{x})$ and
2. loss for $J(\boldsymbol{\theta})$

For $\boldsymbol{\theta} = \{w_1 = \mathbf{0}, w_2 = \mathbf{1}, b = \mathbf{1}\}$

using the data set shown in the table

How to train

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Let's start with a depth 1 model:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{w} \text{ in } \mathbb{R}^2, b \text{ in } \mathbb{R}.$$

The unknown parameters are $\boldsymbol{\theta} = \{w_1, w_2, b\}$

$$J(\theta) = \frac{1}{4} \sum_{i=1}^n (f^*(\mathbf{x}_i) - f(\mathbf{x}_i))^2$$

Exercise:

1. Write the loss function for this dataset
2. Compute the gradient of the loss function.
3. For $\boldsymbol{\theta} = \{w_1=0, w_2=1, b=1\}$ do a gradient step using step size $\eta=0.5$. Is the point $\boldsymbol{\theta} = \{0, 1, 1\}$ a local minimum ?

$$\text{Loss} = J(\boldsymbol{\theta}) = \frac{1}{4} ..$$

How to train

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Let's start with a depth 1 model:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{w} \text{ in } \mathbb{R}^2, b \text{ in } \mathbb{R}.$$

The unknown parameters are $\theta = \{w1, w2, b\}$

$$J(\theta) = \frac{1}{4} \sum_{i=1}^4 (f^*(\mathbf{x}_i) - f(\mathbf{x}_i))^2$$

Exercise:

1. Write the loss function for this dataset
2. Compute the gradient of the loss function.
3. For $\theta = \{w1=\mathbf{0}, w2=\mathbf{1}, b=\mathbf{1}\}$ do a gradient step using step size $\eta=0.5$. Is the point $\theta = \{\mathbf{0}, \mathbf{1}, \mathbf{1}\}$ a local minimum ?

$$\text{Loss} = J(\theta) = \frac{1}{4} ((0 - ((0,0) \cdot (w1, w2) + b))^2 + \dots$$

How to train

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Let's start with a depth 1 model:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{w} \text{ in } \mathbb{R}^2, b \text{ in } \mathbb{R}.$$

1. Write the loss function

$$\begin{aligned} \text{Loss} = J(\theta) = \frac{1}{4} & \left((0 - (0,0) \cdot (\mathbf{w}_1, \mathbf{w}_2) + b))^2 \right. \\ & + (1 - (0,1) \cdot (\mathbf{w}_1, \mathbf{w}_2) + b))^2 + \\ & + (1 - (1,0) \cdot (\mathbf{w}_1, \mathbf{w}_2) + b))^2 + \\ & \left. + (0 - (1,1) \cdot (\mathbf{w}_1, \mathbf{w}_2) + b))^2 \right). \end{aligned}$$

Compute the gradient of the loss
and move the model !

How to train

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Let's start with a depth 1 model:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{w} \text{ in } \mathbb{R}^2, b \text{ in } \mathbb{R}.$$

1. Write the loss function

$$\begin{aligned} \text{Loss} = J(\mathbf{w}) = \frac{1}{4} & \left((0 - ((0,0) * (\mathbf{w}_1, \mathbf{w}_2) + b))^2 \right. \\ & + (1 - ((0,1) * (\mathbf{w}_1, \mathbf{w}_2) + b))^2 + \\ & + (1 - ((1,0) * (\mathbf{w}_1, \mathbf{w}_2) + b))^2 + \\ & \left. + (0 - ((1,1) * (\mathbf{w}_1, \mathbf{w}_2) + b))^2 \right). \end{aligned}$$

$$\begin{aligned} = \frac{1}{4} & \left((0 - b)^2 \right. \\ & + (1 - \mathbf{w}_2 - b)^2 + \\ & + (1 - \mathbf{w}_1 - b)^2 + \\ & \left. + (0 - \mathbf{w}_1 - \mathbf{w}_2 - b)^2 \right) \end{aligned}$$

How to train

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Let's start with a depth
1 model:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{w} \text{ in } \mathbb{R}^2, b \text{ in } \mathbb{R}. \quad J(w_1, w_2, b) = \frac{1}{4} \left[\sum_{i=1}^n (y_i - f(x_i))^2 \right]$$

1. Write the loss
function

$$\begin{aligned} \text{Loss} = J(\mathbf{w}) = &= \frac{1}{4} [(0 - (0w_1 + 0w_2 + b))^2 \\ &+ (1 - (0w_1 + 1w_2 + b))^2 + \\ &(0 - (0w_1 + 1w_2 + b))^2 + (1 - (1w_1 + 1w_2 + b))^2] \end{aligned}$$

How to train

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Let's start with a depth
1 model:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{w} \text{ in } \mathbb{R}^2, b \text{ in } \mathbb{R}. \quad J(w_1, w_2, b) = \frac{1}{4} \left[\sum_{i=1}^n (y_i - f(x_i))^2 \right]$$

1. Write the loss
function

$$\begin{aligned} \text{Loss} = J(\mathbf{w}) &= \\ &= \frac{1}{4} [(0 - b))^2 \\ &\quad + (1 - w_2 - b))^2 + \\ &\quad (1 - w_1 - b))^2 + (0 - w_1 - w_2 - b))^2] \end{aligned}$$

How to train

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Let's start with a depth 1 model:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{w} \text{ in } \mathbb{R}^2, b \text{ in } \mathbb{R}. \quad J(w_1, w_2, b) = \frac{1}{4} \left[\sum_{i=1}^n (y_i - f(x_i))^2 \right]$$

1. Write the loss function

$$\begin{aligned} \text{Loss} = J(\mathbf{w}) &= \frac{1}{4} [b^2 \\ &\quad + (1 - w_2 - b)^2 + \\ &\quad (1 - w_1 - b)^2 + (0 - w_1 - w_2 - b)^2] \end{aligned}$$

Now find the gradient of the loss

let's work on an example

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Let's start with a depth 1 model:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{w} \text{ in } \mathbb{R}^2, b \text{ in } \mathbb{R}.$$

The unknown parameters are $\mathbf{w} = \{w_1, w_2, b\}$
 $d=3, n=4$.

The gradient is a vector.
first coordinate is ...?

let's work on an example

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Let's start with a depth 1 model:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{w} \text{ in } \mathbb{R}^2, b \text{ in } \mathbb{R}.$$

The gradient is a vector.

first coordinate is dJ / dw_1

Second is dJ / dw_2

Third is dJ / db

Therefore first coordinate of gradient is computed:

$$\frac{d}{dw_1} \frac{1}{4} [b^2 + (1 - w_2 - b)^2 + (1 - w_1 - b)^2 + (-w_1 - w_2 - b)^2]$$

let's work on an example

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Let's start with a depth 1 model:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \mathbf{w} \text{ in } \mathbb{R}^2, b \text{ in } \mathbb{R}.$$

The gradient is a vector.

first coordinate is dJ / dw_1

Second is dJ / dw_2

Third is dJ / db

Therefore first coordinate of gradient is computed:

$$\frac{d}{dw_1} \frac{1}{4} [b^2 + (1 - w_2 - b)^2 + (1 - w_1 - b)^2 + (-w_1 - w_2 - b)^2]$$
$$\frac{1}{4} \left[\frac{d}{dw_1} b^2 + \frac{d}{dw_1} (1 - w_2 - b)^2 + \frac{d}{dw_1} (1 - w_1 - b)^2 + \frac{d}{dw_1} (-w_1 - w_2 - b)^2 \right]$$

$$\frac{1}{4} [0 + 0 + -2(1 - w_1 - b) + -2(-w_1 - w_2 - b)] = \frac{1}{2} (2w_1 + w_2 + 2b - 1)$$

let's work on an example

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Gradient at (0,1,1)=

Let's start with a depth 1 model:
 $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$, $\mathbf{w}$ in $\mathbb{R}^2$, b in $\mathbb{R}$.
The unknown parameters are $\theta = \{w_1, w_2, b\}$
 $d=3$, $n=4$.

$$\text{Loss} = J(\theta) = \frac{1}{4} \left((0 - b)^2 + (1 - w_2 - b)^2 + (1 - w_1 - b)^2 + (0 - w_1 - w_2 - b)^2 \right)$$

The gradient is a vector.
first coordinate is dJ / dw_1
Second is $dJ / dw_2 = 1/2$
Third is dJ / db

$2w_1 + w_2 + 2b - 1$
$w_1 + 2w_2 + 2b - 1$
$2w_1 + 2w_2 + 4b - 2$

let's work on an example

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Gradient at (0,1,1)=

1
3/2
2

Let's start with a depth 1 model:
 $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$, $\mathbf{w}$ in $\mathbb{R}^2$, b in $\mathbb{R}$.
The unknown parameters are $\theta = \{w_1, w_2, b\}$
 $d=3$, $n=4$.

$$\text{Loss} = J(\theta) = \frac{1}{4} \left((0 - b)^2 + (1 - w_2 - b)^2 + (1 - w_1 - b)^2 + (0 - w_1 - w_2 - b)^2 \right)$$

The gradient is a vector.
first coordinate is dJ / dw_1
Second is $dJ / dw_2 = 1/2$
Third is dJ / db

$2w_1 + w_2 + 2b - 1$
$w_1 + 2w_2 + 2b - 1$
$2w_1 + 2w_2 + 4b - 2$

let's work on an example

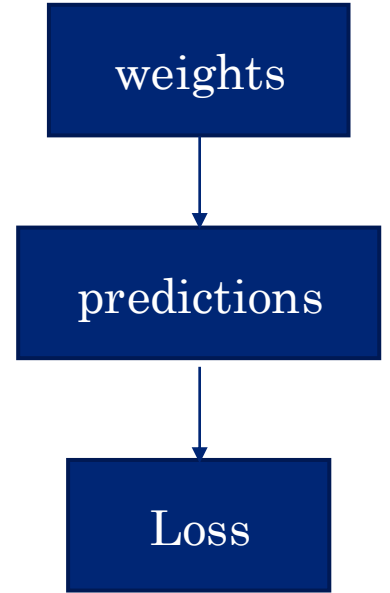
x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Gradient at $(0,1,1) = \nabla J(\mathbf{w}) =$

$$\text{Loss} = J(\mathbf{w}) = \frac{1}{4} \left((0 - b)^2 + (1 - w_2 - b)^2 + (1 - w_1 - b)^2 + (0 - w_1 - w_2 - b)^2 \right)$$

The gradient is a vector.
 first coordinate is dJ / dw_1
 Second is $dJ / dw_2 = 1/2$
 Third is dJ / db

1
3/2
2



$2w_1 + w_2 + 2b - 1$
$w_1 + 2w_2 + 2b - 1$
$2w_1 + 2w_2 + 4b - \mathbf{2}$

Gradient step with LR=0.5

$$\mathbf{w}^{(1)} = \mathbf{w}^{(0)} - \eta \nabla J(\mathbf{w}^{(0)})$$

$$\begin{bmatrix} -1/2 \\ 1/4 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} - \eta \begin{bmatrix} 1 \\ 1.5 \\ 2 \end{bmatrix}$$

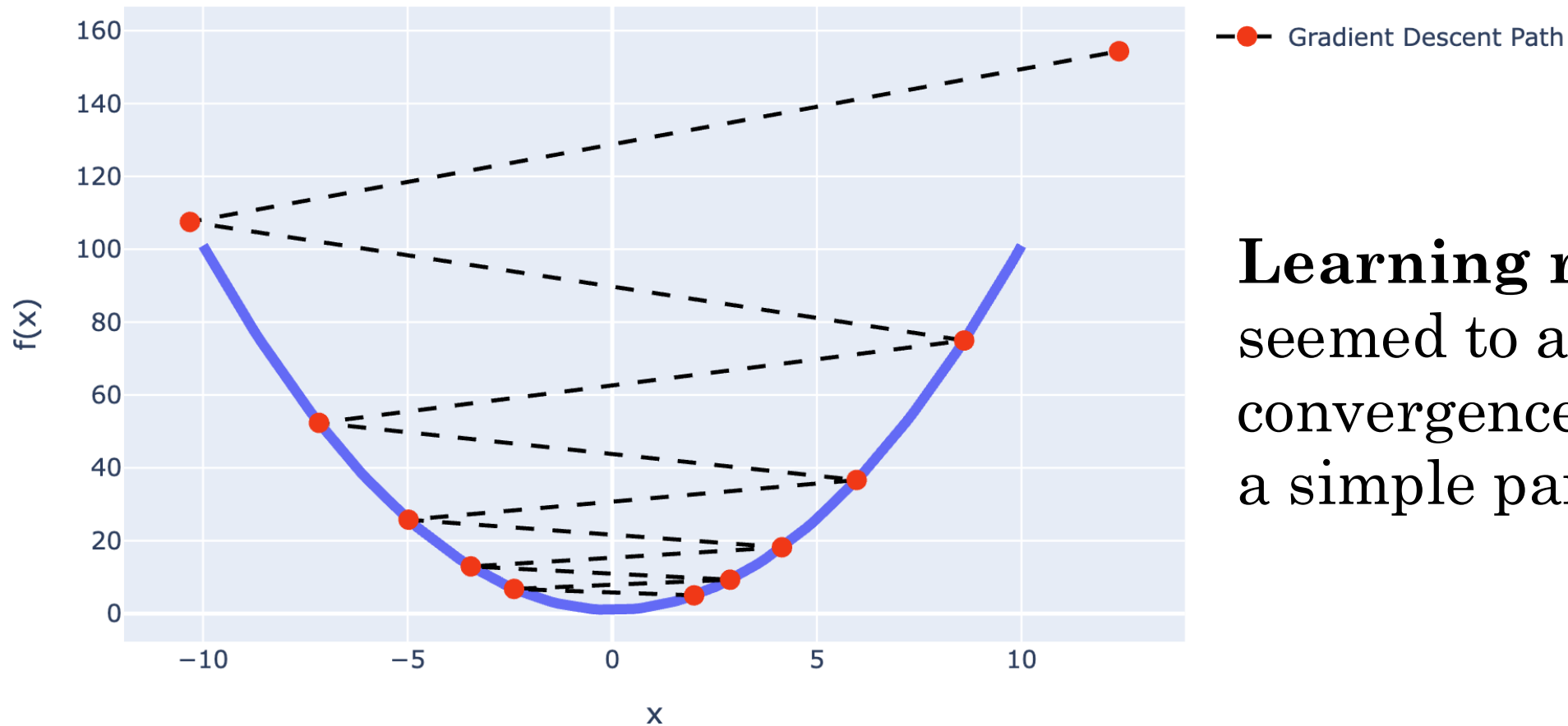
$$J(\mathbf{w}^{(0)}) = 1.5$$

$$J(\mathbf{w}^{(1)}) = \frac{1}{4} * \left(0 + \left(-\frac{3}{4} \right) \right)^2 + (0 - 3/2)^2 + (0 - 1/4)^2 = 0.718$$

$$\mathbf{w}^{(0)} = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$$

$$\nabla J(\mathbf{w}^{(0)}) = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$$

What are the conditions for convergence?



Learning rate
seemed to affect
convergence even on
a simple parabola.

Convergence Assuming Quadratic Approximation

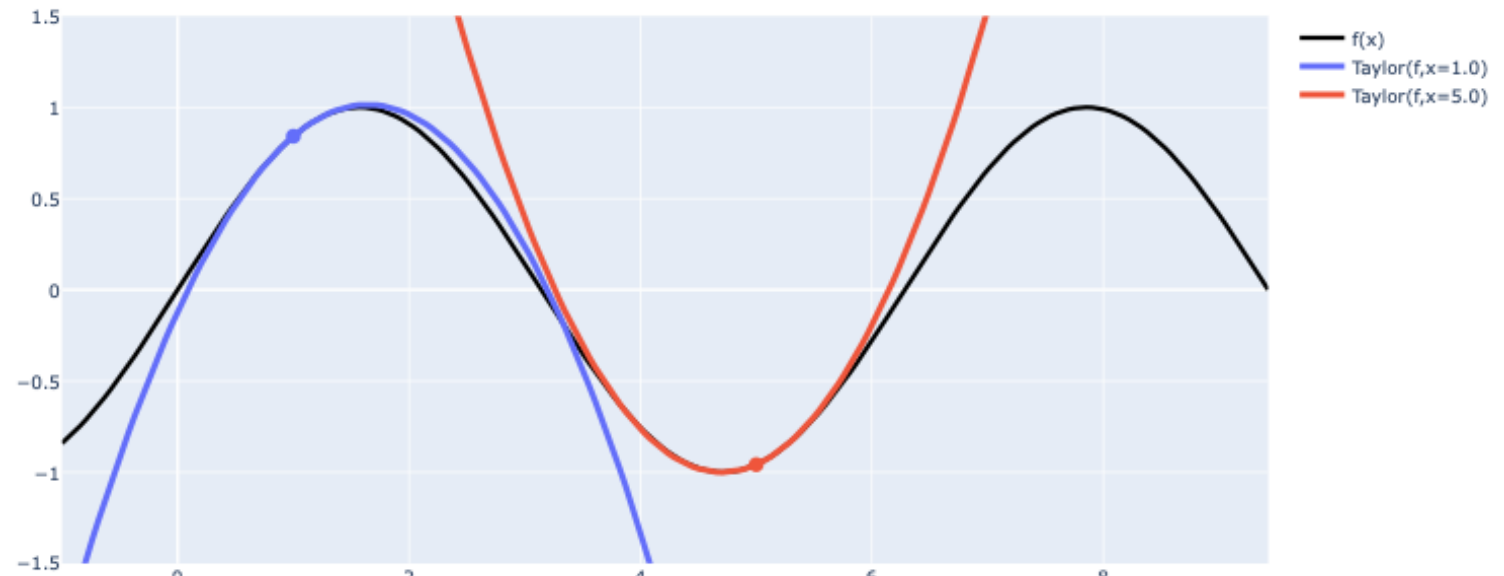
Quadratic Approximations

The 2nd order (quadratic) Taylor approximation of a function $f: \mathbb{R} \rightarrow \mathbb{R}$ evaluated at $\hat{x}$ is:

$$\tilde{f}(x) = f(\hat{x}) + \left(\frac{\partial}{\partial x} f(x) \Big|_{x=\hat{x}} \right) (x - \hat{x}) + \frac{1}{2} \left(\frac{\partial^2}{\partial x^2} f(x) \Big|_{x=\hat{x}} \right) (x - \hat{x})^2$$

For example: $f(x) = \sin(x)$

$$\begin{aligned} \tilde{f}_{\hat{x}=1}(x) &= \sin(1) + \cos(1)(x - 1) \\ &\quad + \frac{1}{2}(-\sin(1))(x - 1)^2 \\ \tilde{f}_{\hat{x}=5}(x) &= \sin(5) + \cos(5)(x - 5) \\ &\quad + \frac{1}{2}(-\sin(5))(x - 5)^2 \end{aligned}$$



Taylor Expansion of the Error Surface

The 2nd-order Taylor expansion of the **error surface** at $\hat{w}$:

$$\tilde{E}_{\hat{w}}(w) = E(\hat{w}) + (w - \hat{w})^T b + \frac{1}{2} (w - \hat{w})^T H (w - \hat{w})$$

- $b = \nabla E(\hat{w})$ is the **Gradient** of E evaluated at $\hat{w}$ (vector)
- $H = \nabla \nabla E(\hat{w})$ is the **Hessian** of E evaluated at $\hat{w}$ (a matrix)

Matrix Equivalent of
the Second Derivative

Hessians (the 2nd derivative)

The Hessian of a **scalar-valued function** $E: \mathbb{R}^D \rightarrow \mathbb{R}$ is the **matrix-valued function** $\nabla^2 E: \mathbb{R}^D \rightarrow \mathbb{R}^{D \times D}$ of the 2nd partial derivatives of f

$$H = \nabla^2 E = \begin{bmatrix} \frac{\partial^2}{\partial x_1^2} E(x) & \dots & \frac{\partial^2}{\partial x_1 x_D} E(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial^2}{\partial x_D x_1} E(x) & \dots & \frac{\partial^2}{\partial x_D^2} E(x) \end{bmatrix}$$

The Hessian

- describes the local **curvature** of E
- determines the **critical point type** (minima, maxima, or saddle)

Hessian Exercise

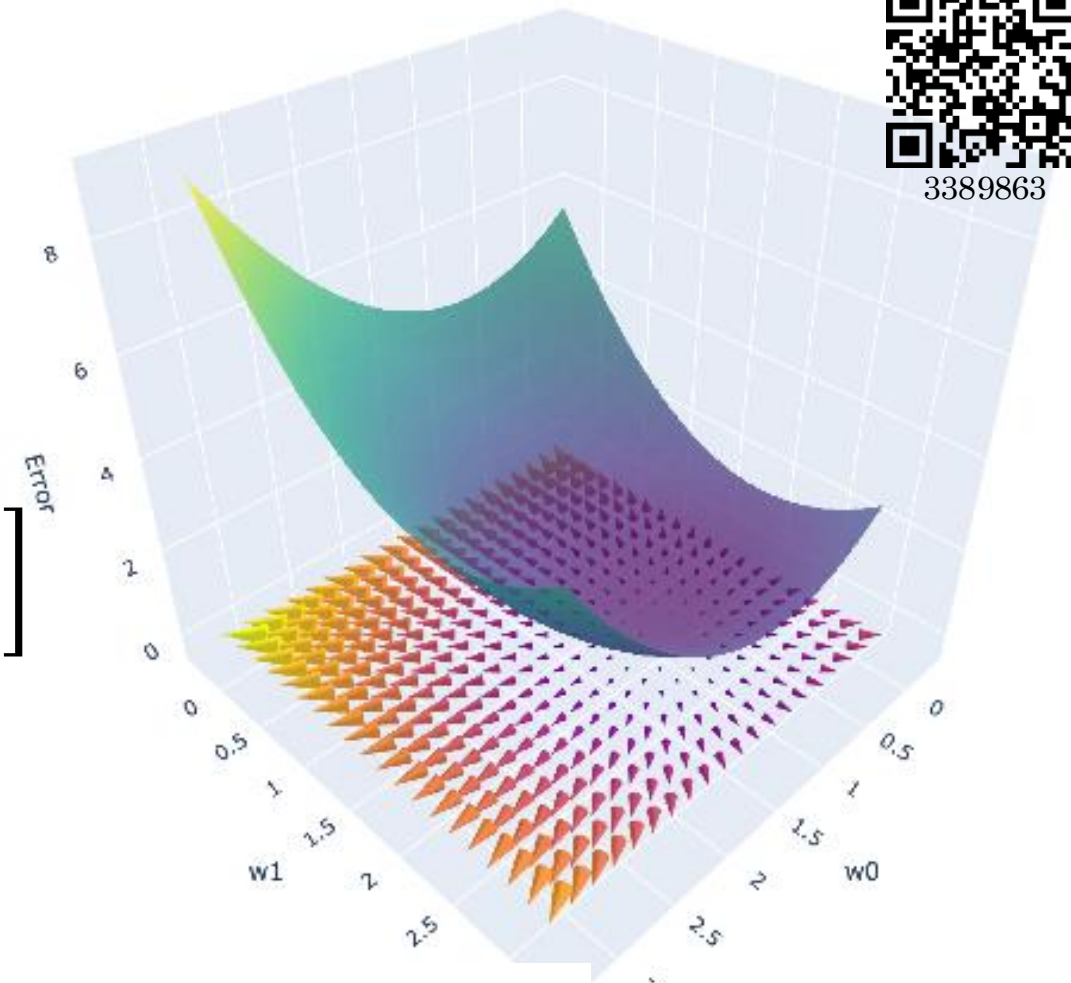
Consider the function:

$$E(w) = (w_0 - 1)^2 + (w_1 - 2)^2 + 1$$

Then the gradient is: $\nabla E(w) = \begin{bmatrix} 2(w_0 - 1) \\ 2(w_1 - 2) \end{bmatrix}$

The Hessian is:

$$H(w) = \begin{bmatrix} \frac{\partial^2}{\partial w_0^2} E(w) & \frac{\partial^2}{\partial w_0 \partial w_1} E(w) \\ \frac{\partial^2}{\partial w_0 \partial w_1} E(w) & \frac{\partial^2}{\partial w_1^2} E(w) \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$



Hessian

$$\nabla^2 E = \begin{bmatrix} \frac{\partial^2}{\partial x_1^2} E(x) & \dots & \frac{\partial^2}{\partial x_1 \partial x_D} E(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial^2}{\partial x_D \partial x_1} E(x) & \dots & \frac{\partial^2}{\partial x_D^2} E(x) \end{bmatrix}$$



3389863

Taylor Expansion of the Error Surface

The 2nd-order Taylor expansion of the error surface at $\hat{w}$:

$$\tilde{E}_{\hat{w}}(w) = E(\hat{w}) + (w - \hat{w})^T b + \frac{1}{2} (w - \hat{w})^T H (w - \hat{w})$$

- $b = \nabla E(\hat{w})$ is the **Gradient** of E evaluated at $\hat{w}$ (vector)
- $H = \nabla \nabla E(\hat{w})$ is the **Hessian** of E evaluated at $\hat{w}$ (a matrix)

If $\hat{w}$ is a **critical point** (w^*) then $\nabla E(w^*) = 0 = b$

$$\tilde{E}_{w^*}(w) = E(w^*) + \frac{1}{2} (w - w^*)^T H (w - w^*)$$

Use a change of variables to better understand this part.

Eigen Decomposition of the Hessian

We can construct the **Eigen decomposition of the Hessian**

$$Hu_i = \lambda_i u_i$$

- u_i are the complete set of **orthonormal eigenvectors**: $\forall_{i,j}: u_i^T u_j = \delta_{ij}$
- λ_i are the real-valued **eigenvalues**

Then we apply a **change of variables**:

$$w - w^* = \sum_i \alpha_i u_i \quad \text{where} \quad \alpha_i = (w - w^*)^T u_i$$

This allows us to rewrite:

$$\tilde{E}_{w^*}(w) = E(w^*) + \frac{1}{2} (w - w^*)^T H (w - w^*) = E(w^*) + \frac{1}{2} \sum_i \alpha_i^2 \lambda_i$$

Show it!

- u_i are the complete set of **orthonormal eigenvectors**: $\forall_{i,j}: u_i^T u_j = \delta_{ij}$
- λ_i are the real-valued **eigenvalues**

Then we apply a **change of variables**:

$$w - w^* = \sum_i \alpha_i u_i \quad \text{where} \quad \alpha_i = (w - w^*)^T u_i$$

This allows us to rewrite:

$$\tilde{E}_{w^*}(w) = E(w^*) + \frac{1}{2} (w - w^*)^T H (w - w^*) = E(w^*) + \frac{1}{2} \sum_i \alpha_i^2 \lambda_i$$

Show it!

Derivation:

$$\begin{aligned} (w - w^*)^T H (w - w^*) &= \left(\sum_i \alpha_i u_i \right)^T H \left(\sum_j \alpha_j u_j \right) = \left(\sum_i \alpha_i u_i \right)^T \left(\sum_j \alpha_j H u_j \right) \\ &= \left(\sum_i \alpha_i u_i \right)^T \left(\sum_j \alpha_j \lambda_j u_j \right) = \sum_{ij} \alpha_i \alpha_j u_i^T u_j \lambda_j = \sum_{ij} \alpha_i \alpha_j \delta_{ij} \lambda_j = \sum_i \alpha_i^2 \lambda_i \end{aligned}$$

Eigenvector Defn.

Orthonormality

Taylor Expansion at the Stationary Pts.

If $\hat{w}$ is a **stationary point** (w^*) (where $b = \nabla E|_{w=\hat{w}} = 0$)

$$\tilde{E}_{w^*}(\alpha) = E(w^*) + \frac{1}{2} \sum_i \alpha_i^2 \lambda_i$$

At $\alpha = 0$ we get the error at the critical point $E(w^*)$.

- If we move in **direction i** (corresponding to u_i) then α_i^2 **increases**.
 - The change in error then depends on the sign of λ_i

Therefore, if the **eigenvalues are ..?**

(what happens at local minimum) and Local Maximum and at saddle point?

Taylor Expansion at the Stationary Pts.

If $\hat{w}$ is a **stationary point** (w^*) (where $b = \nabla E|_{w=\hat{w}} = 0$)

$$\tilde{E}_{w^*}(\alpha) = E(w^*) + \frac{1}{2} \sum_i \alpha_i^2 \lambda_i$$

At $\alpha = 0$ we get the error at the critical point $E(w^*)$.

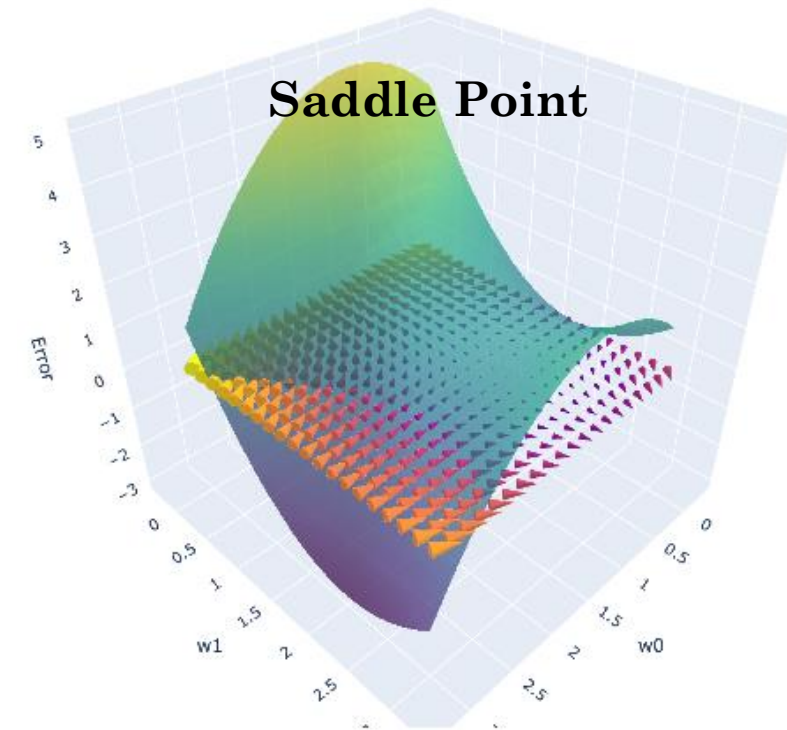
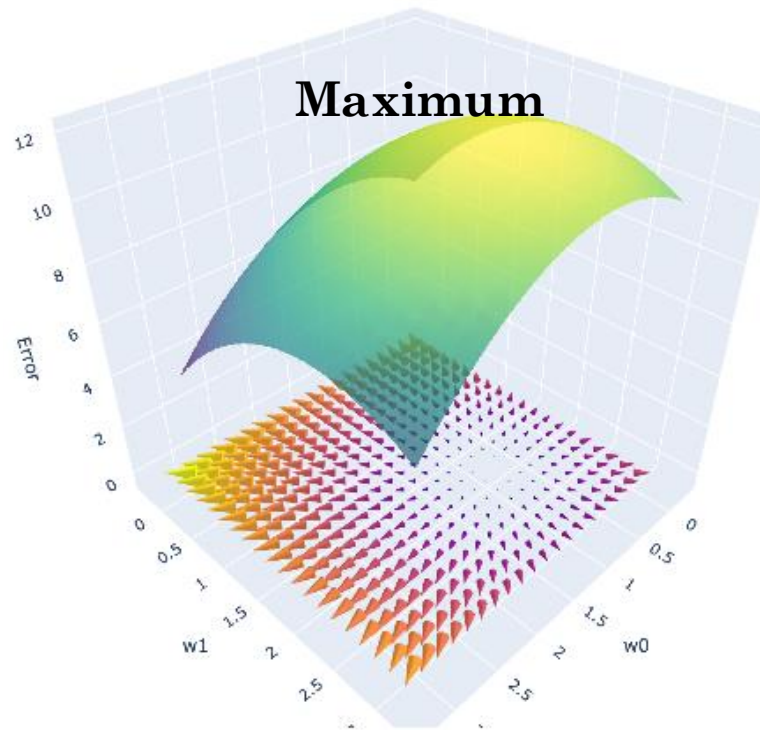
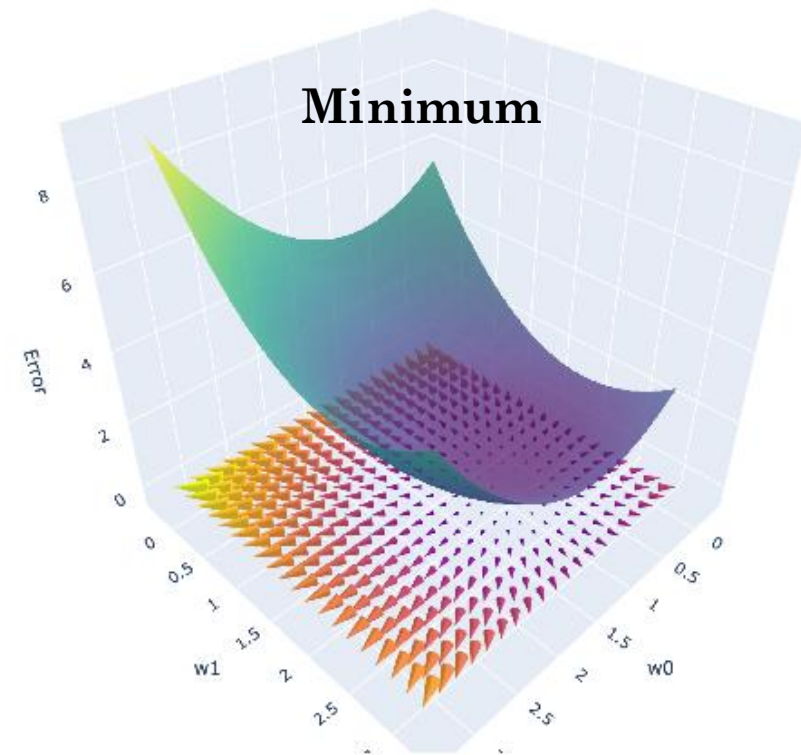
- If we move in **direction** i (corresponding to u_i) then α_i^2 **increases**.
 - The change in error then depends on the sign of λ_i

Therefore, if the **eigenvalues** are

- **all positive** then w^* is a **local minimum**.
- **all negative** then w^* is a **local maximum**.
- **mixed** then w^* is a **saddle point**.

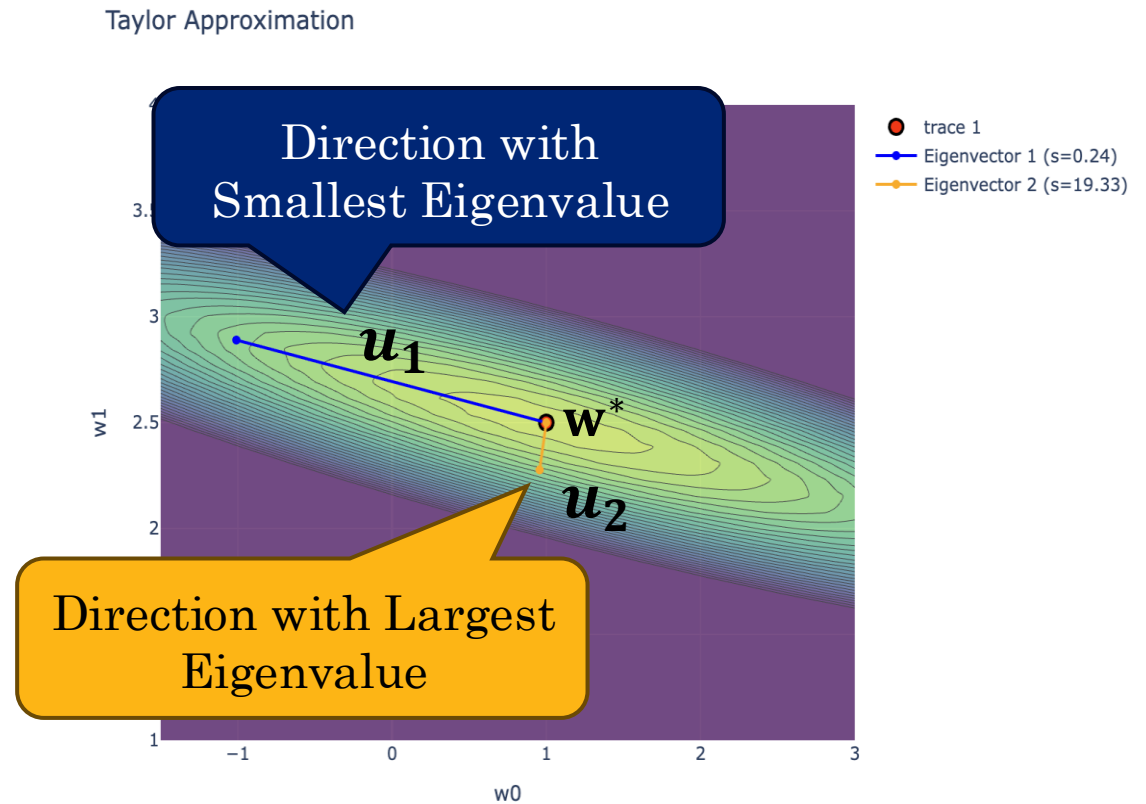
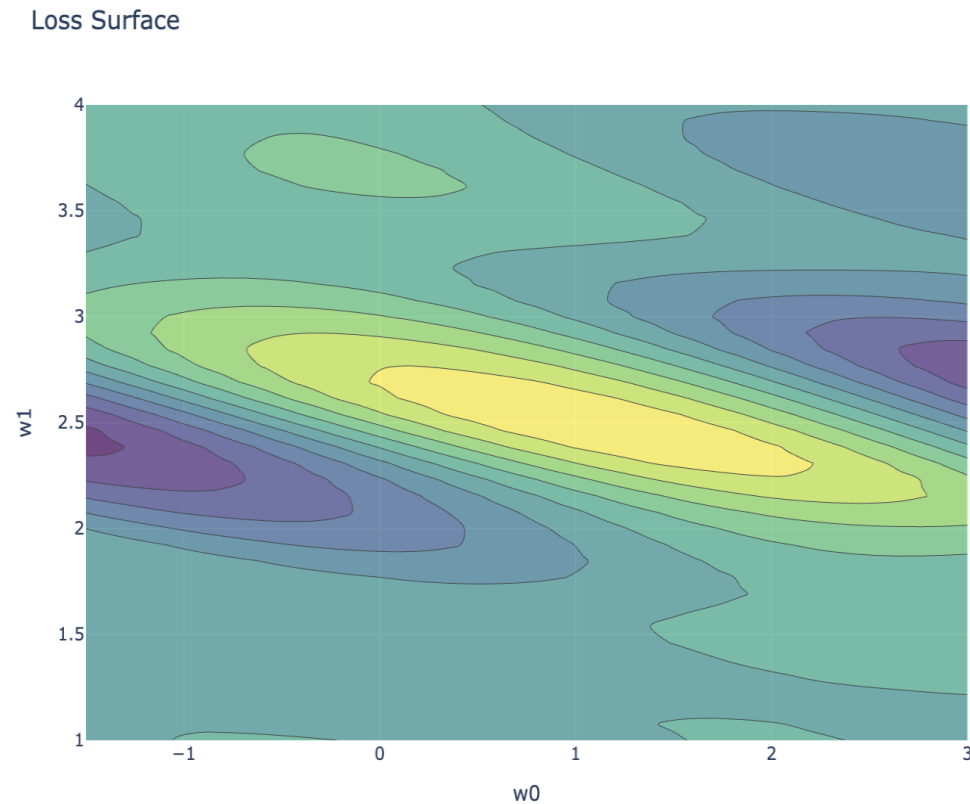
Stationary Points and the Hessian

Eigenvalues of the Hessian determine the curvature at the critical points



Eigenvectors of the Hessian

Elliptical contours of constant error align with the eigenvectors of the Hessian matrix.



Recap: Quadratic Approx. Error

We constructed the **2nd-order Taylor expansion** of the error surface at the **stationary point** w^* (where $b = \nabla E|_{w=w^*} = 0$):

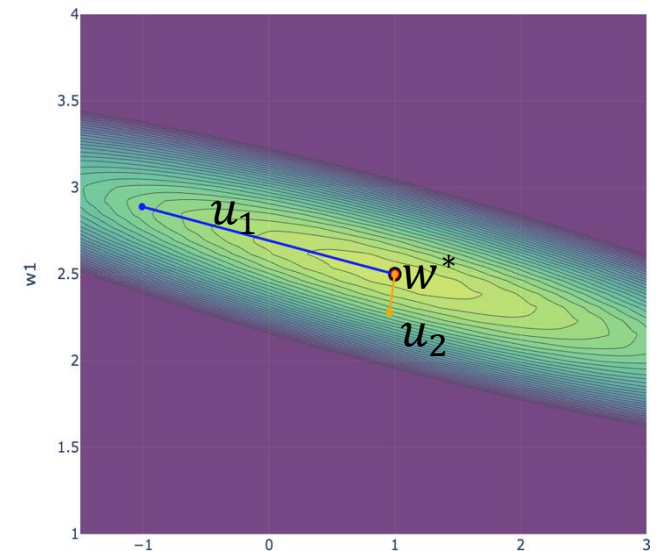
$$\tilde{E}_{w^*}(w) = E(w^*) + \frac{1}{2} (w - w^*)^T H (w - w^*)$$

Using the **Eigen-decomposition of the Hessian**: $Hu_i = \lambda_i u_i$ we applied a change of variables $\alpha_i = (w - w^*)^T u_i$ to obtain:

$$\tilde{E}_{w^*}(\alpha) = E(w^*) + \frac{1}{2} \sum_i \alpha_i^2 \lambda_i$$

- This is minimized at $\alpha = 0$

We can **evaluate gradient descent** on this simple quadratic approximation to better understand the convergence properties.



Grad. Descent on the Quadratic Approx.

Compute the **gradient** with respect to α_i :

$$\frac{\partial}{\partial \alpha_i} \tilde{E}_{w^*}(\alpha) = \alpha_i \lambda_i$$

Quadratic Approx. Error

$$\tilde{E}_{w^*}(\alpha) = E(w^*) + \frac{1}{2} \sum_i \alpha_i^2 \lambda_i$$

Which gives the **gradient descent update**:

$$\alpha_i^{(\tau)} = \alpha_i^{(\tau-1)} - \eta \left(\alpha_i^{(\tau-1)} \lambda_i \right) = (1 - \eta \lambda_i) \alpha_i^{(\tau-1)}$$

If we **iteratively** apply the gradient descent update, we obtain:

$$\alpha_i^{(1)} = (1 - \eta \lambda_i) \alpha_i^{(0)} \Rightarrow \alpha_i^{(2)} = (1 - \eta \lambda_i) \left((1 - \eta \lambda_i) \alpha_i^{(0)} \right) \Rightarrow \dots \Rightarrow$$

$$\alpha_i^{(\tau)} = (1 - \eta \lambda_i)^\tau \alpha_i^{(0)}$$

Grad. Descent on the Quadratic Approx.

Assuming a **quadratic approximation** to the error function around the minima, the τ^{th} iteration of gradient descent we have:

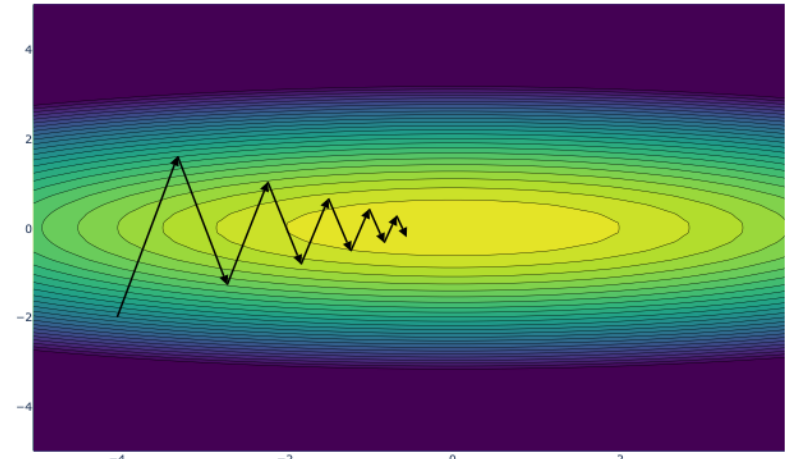
$$\alpha_i^{(\tau)} = (1 - \eta\lambda_i)^\tau \alpha_i^{(0)}$$

and the **error is minimized when $\alpha_i = 0$** .

To ensure $\alpha_i^{(\tau)} \rightarrow 0$ as $\tau \rightarrow \infty$ the learning rate should satisfy:

$$|1 - \eta\lambda_i| < 1 \quad (\text{Convergence Condition})$$

- Smaller $|1 - \eta\lambda_i|$ is better (converges faster)
- Negative $1 - \eta\lambda_i < 0$ oscillatory behavior



Learning Rate Impact on Convergence

To maximize the **progress on each step** of gradient descent we want to **make η as large as possible (take bigger steps)**

- We want $(1 - \eta\lambda_i)^\tau \rightarrow 0$ quickly as τ increases (larger $\eta \rightarrow$ smaller $1 - \eta\lambda_i$).
- However, to **ensure convergence we require $|1 - \eta\lambda_i| < 1$ for all λ_i**

Let $\lambda_{\min}$ and $\lambda_{\max}$ be the smallest and largest **eigenvalues** of the Hessian

- Then $\eta < \frac{2}{\lambda_{\max}}$ because if $\eta \geq \frac{2}{\lambda_{\max}}$ then $|1 - \eta\lambda_{\max}| \geq 1$

The **slowest converging dimension** correspond to $\lambda_{\min}$ so the smallest we can make $(1 - \eta\lambda_i)$:

$$\left(1 - \frac{2}{\lambda_{\max}}\lambda_{\min}\right) < (1 - \eta\lambda_i)$$

Condition Number and Loss Surface

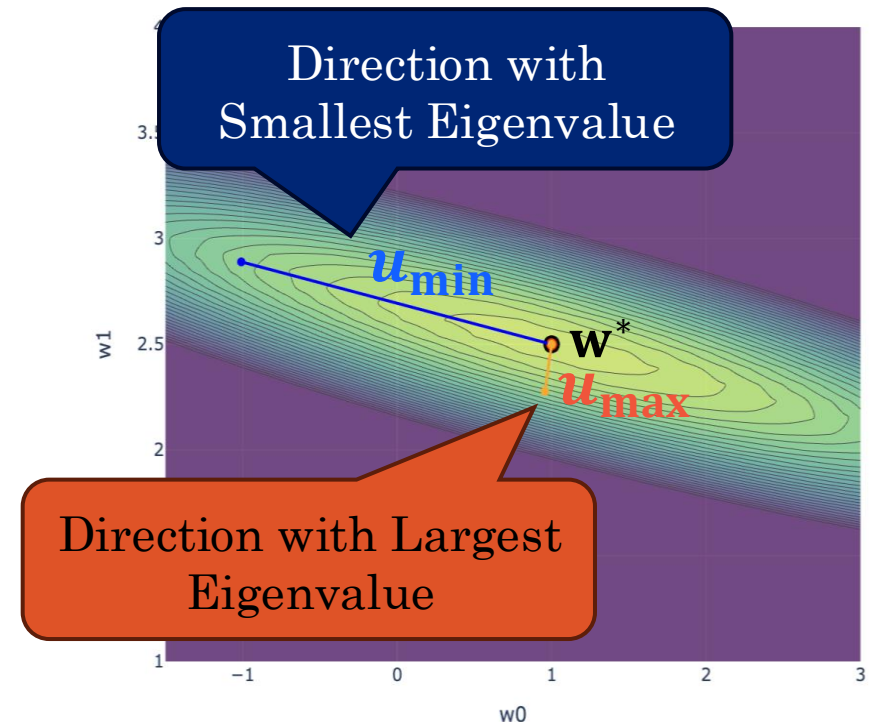
The slowest converging dimension (i) is the dimension corresponding to $\lambda_{\min}$ and its convergence to 0 is lower bounded by:

$$\left(1 - \frac{2}{\lambda_{\max}} \lambda_{\min}\right)^{\tau} \alpha_i^{(0)}$$

The ratio of $\frac{\lambda_{\min}}{\lambda_{\max}}$ is the **reciprocal** of the **condition number**.

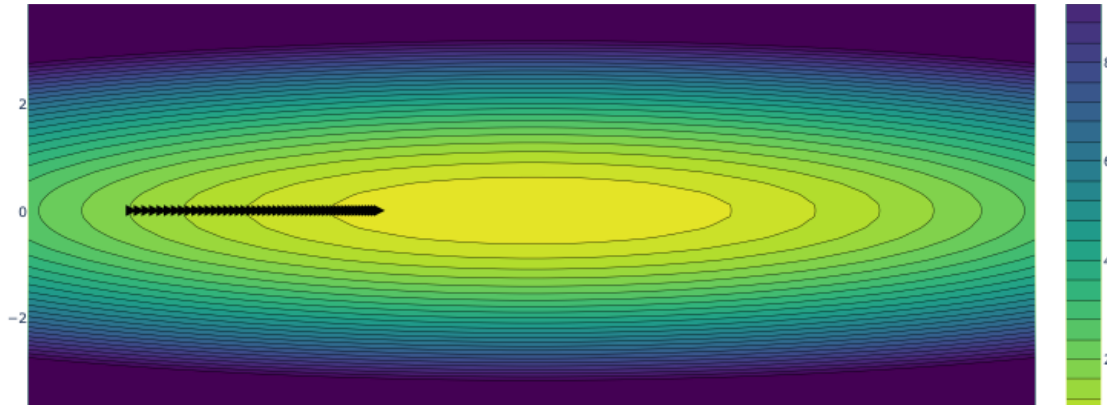
- small $\frac{\lambda_{\min}}{\lambda_{\max}}$ implies slow convergence

Gradient descent converges quickly on **well conditioned problems**.



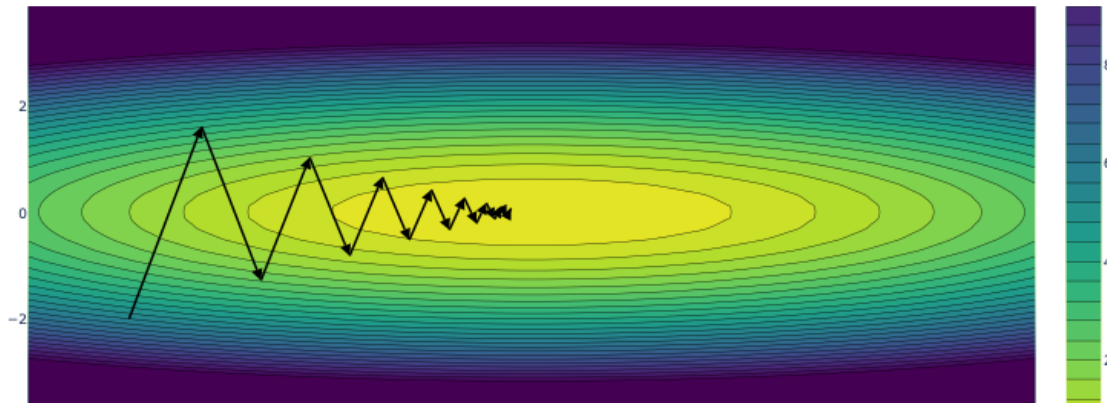
Issues with Gradient Descent (GD)

GD converges slowly when the **error surface is relatively flat**:



Step size
(learning rate η)
is **too small**.

GD oscillates when the error surface is **poorly conditioned**



Step size
(learning rate η)
is **too big**.

Momentum

Momentum can be used improve convergence in these settings by **accumulating common update directions**

$$\Delta \mathbf{w}^{(\tau-1)} = -\eta \nabla J(\mathbf{w}^{(\tau-1)}) + \mu \Delta \mathbf{w}^{(\tau-2)}$$

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} + \Delta \mathbf{w}^{(\tau-1)}$$

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta \nabla J(\mathbf{w}^{(\tau-1)}) + \mu (\mathbf{w}^{(\tau-1)} - \mathbf{w}^{(\tau-2)})$$

- $0 \leq \mu < 1$ ($\mu = 0.9$) is the **momentum hyperparameter**

We now need to maintain the previous $\Delta \mathbf{w}^{(\tau-2)}$ update direction

Intuition: The solution “**picks up speed**” along the direction of repeated updates.

How Momentum Helps with Flat Dir.

In **directions** that are **relatively flat** (e.g., is approx. constant ∇E):

$$\begin{aligned}\Delta \mathbf{w}^{(\infty)} &= -\eta \nabla J + \mu \left(-\eta \nabla J + \mu \left(-\eta \nabla J + \mu (\dots) \right) \right) \\ \Delta \mathbf{w}^{(\infty)} &= -\eta \nabla J \underbrace{(1 + \mu + \mu^2 + \mu^3 + \dots)}_{\text{Geometric Series}} = -\left(\frac{\eta}{1 - \mu} \right) \nabla J\end{aligned}$$

The **momentum parameter** μ **increases** the effective learning rate

- Remember that $0 \leq \mu < 1$ and therefore $\eta \leq \left(\frac{\eta}{1 - \mu} \right)$

How Momentum Helps with Oscillations

In **directions** that **are steep** (e.g., oscillating sign ∇E):

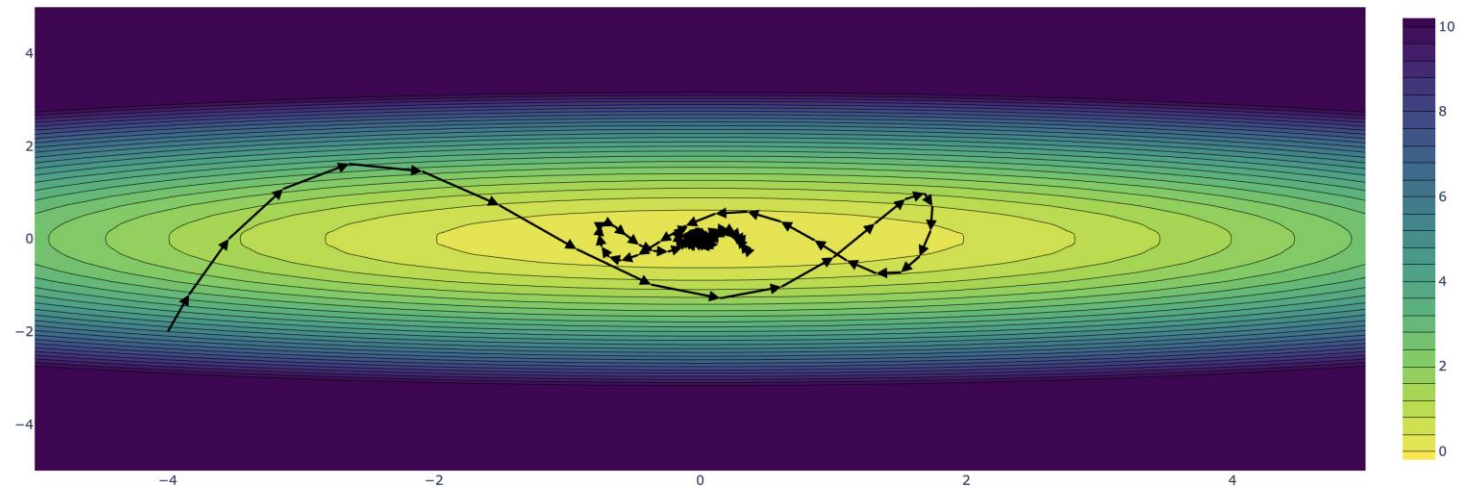
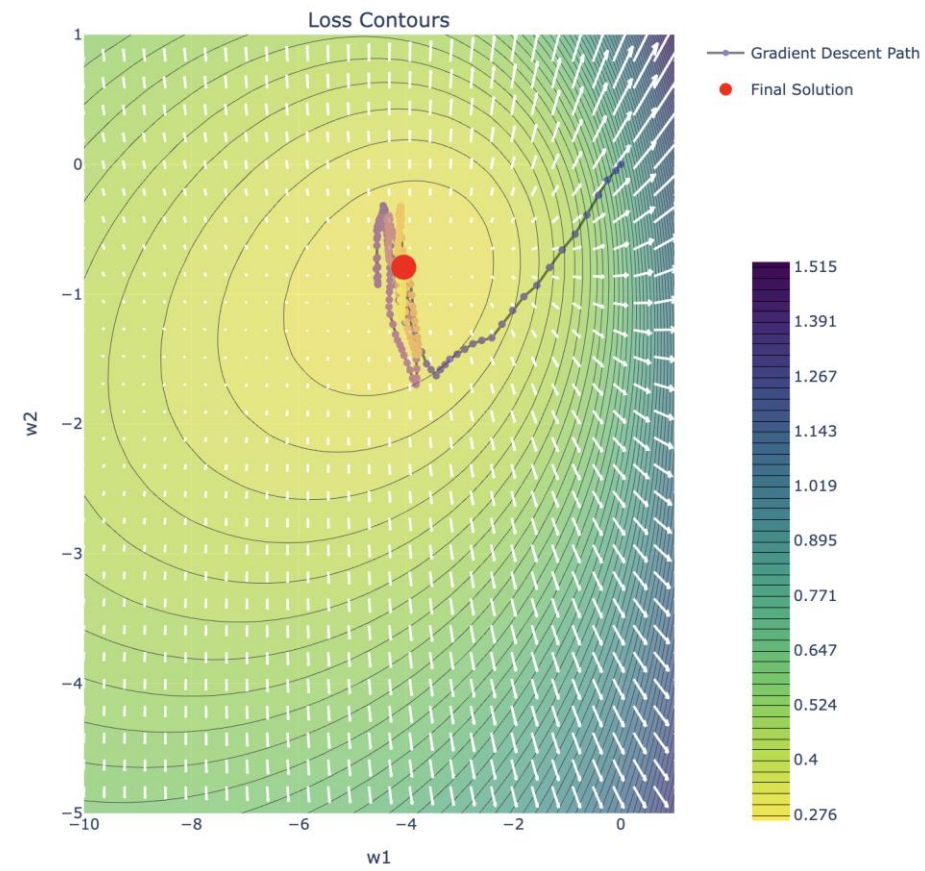
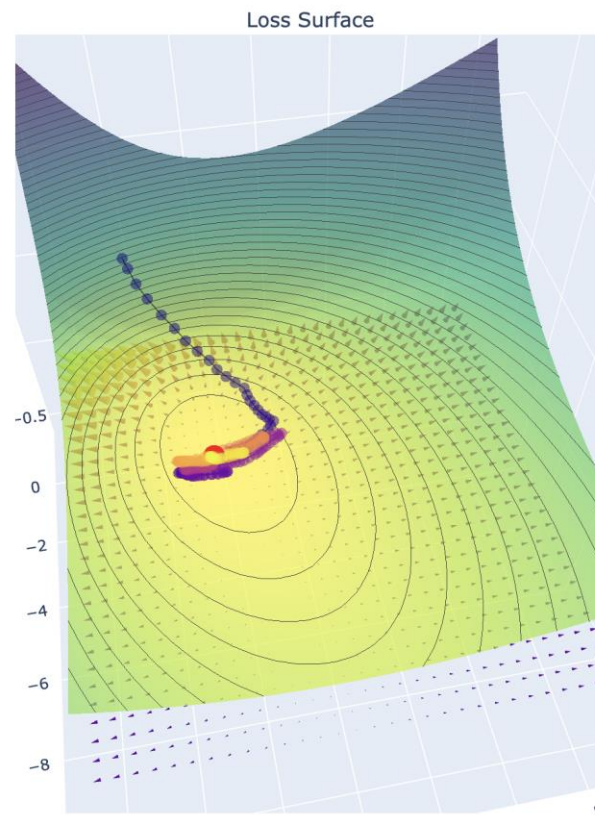
$$\Delta \mathbf{w}^{(\tau)} = -\eta \nabla J + \mu \left(-\eta (-\nabla J) + \mu (-\eta \nabla J + \mu (\dots)) \right)$$
$$\Delta \mathbf{w}^{(\infty)} = -\eta \nabla J \underbrace{(1 - \mu + \mu^2 - \mu^3 + \dots)}_{\text{Alternating Geometric Series}} = -\left(\frac{\eta}{1 + \mu} \right) \nabla J$$

The **momentum parameter** μ **decrease** the effective learning rate

- Remember that $0 \leq \mu < 1$ and therefore $\left(\frac{\eta}{1 + \mu} \right) \leq \eta$

Demo

Momentum



Learning Rate Schedules

A larger initial learning rate can accelerate convergence but to ensure convergence we often reduce the learning rate across iterations.

- **Linear Schedule:** interpolates between η_{start} and η_{end} over T iterations

$$\eta^{(\tau)} = \left(1 - \frac{\tau}{T}\right) \eta_{\text{start}} + \left(\frac{\tau}{T}\right) \eta_{\text{end}}$$

- **Power-law Schedule:** (most common) with η_{start} , s , and c hyperparameters

$$\eta^{(\tau)} = \eta_{\text{start}} \left(1 + \frac{\tau}{s}\right)^{-c}$$

- **Exponential Schedule:** with η_{start} , s , and c hyperparameters

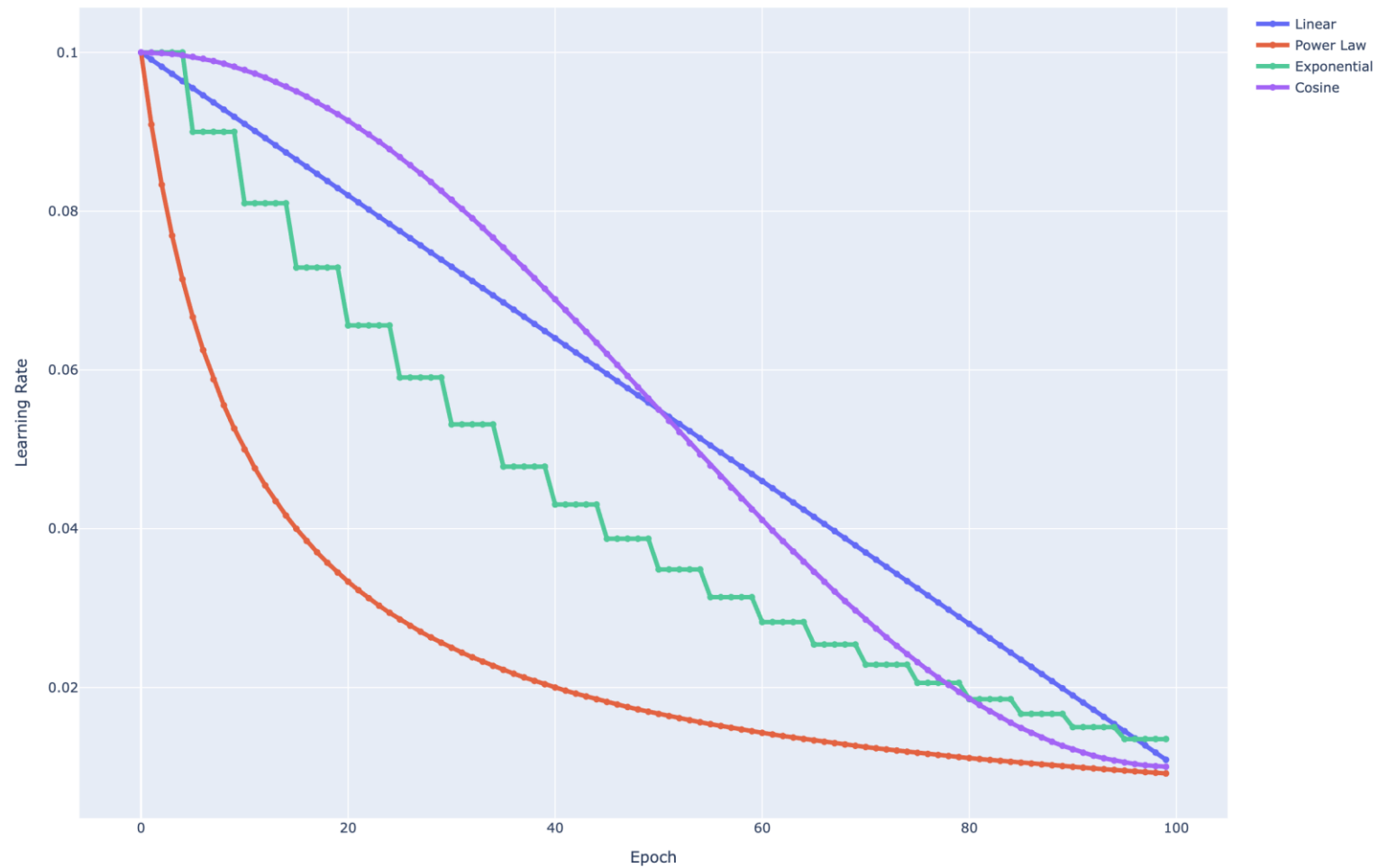
$$\eta^{(\tau)} = \eta_{\text{start}} c^{\lfloor \frac{\tau}{s} \rfloor}$$

- **Cosine Schedule**: common for LLMs, with η_{min} , η_{max} , and T hyperparameters

$$\eta^{(\tau)} = \eta_{\text{min}} + \frac{1}{2} (\eta_{\text{max}} - \eta_{\text{min}}) \left(1 + \cos\left(\frac{\tau}{T} \pi\right)\right)$$

Demo

Learning Rate Schedules



Adaptive Learning Rates – AdaGrad

AdaGrad* (Adaptive Gradient) reduces learning rate on dimensions with greatest curvature (largest gradient magnitude).

Reduce learning rate by running squared average of the gradient.

$$r_i^{(\tau)} = r_i^{(\tau-1)} + \left(\frac{\partial}{\partial w_i} J(\mathbf{w}) \right)^2$$

Tracking Magnitude
(terms squared)

$$w_i^{(\tau)} = w_i^{(\tau-1)} - \frac{\eta}{\sqrt{r_i^{(\tau)} + \epsilon}} \left(\frac{\partial}{\partial w_i} J(\mathbf{w}) \right)$$

Scaled Learning Rate

Issue:

Large initial gradients result in overly small learning rates near convergence.

- $\epsilon \approx 10^{-8}$ is a small constant for numerical stability.

*Developed at UC Berkeley

Adaptive Learning Rates – RMSProp

RMSProp focuses on the **most recent gradients** by replacing the average of the gradients with an **exponential weighted average**

AdaGrad $r_i^{(\tau)} = r_i^{(\tau-1)} + \left(\frac{\partial}{\partial w_i} J(\mathbf{w}) \right)^2$

RMSProp $r_i^{(\tau)} = \beta r_i^{(\tau-1)} + (1 - \beta) \left(\frac{\partial}{\partial w_i} J(\mathbf{w}) \right)^2$

- Initialize $r_i^{(0)} = 0$.
- The hyperparameter β is typically $\beta = 0.9$

Adam = RMSProp + Momentum

Adam (adaptive moments) combines RMSProp with moments.

Momentum $\rightarrow s_i^{(\tau)} = \beta_1 s_i^{(\tau-1)} + (1 - \beta_1) \left(\frac{\delta}{\delta w_i} J(\mathbf{w}) \right)$

RMSProp $\rightarrow r_i^{(\tau)} = \beta_2 r_i^{(\tau-1)} + (1 - \beta_2) \left(\frac{\delta}{\delta w_i} J(\mathbf{w}) \right)^2$

Gradient Update $\rightarrow w_i^{(\tau)} = w_i^{(\tau-1)} - \eta \frac{s_i^{(\tau)}}{\sqrt{r_i^{(\tau)} + \delta}}$

- Initialize $s_i^{(0)} = 0$ and $r_i^{(0)} = 0$.
- The **hyperparameters** β_1 and β_2 are typically $\beta_1 = 0.9$ and $\beta_2 = 0.99$.

Adam is probably the most widely used gradient descent update.

Bias Corrected Version:

$$\hat{s}_i^{(\tau)} = \frac{s_i^{(\tau)}}{1 - \beta_1^\tau}$$

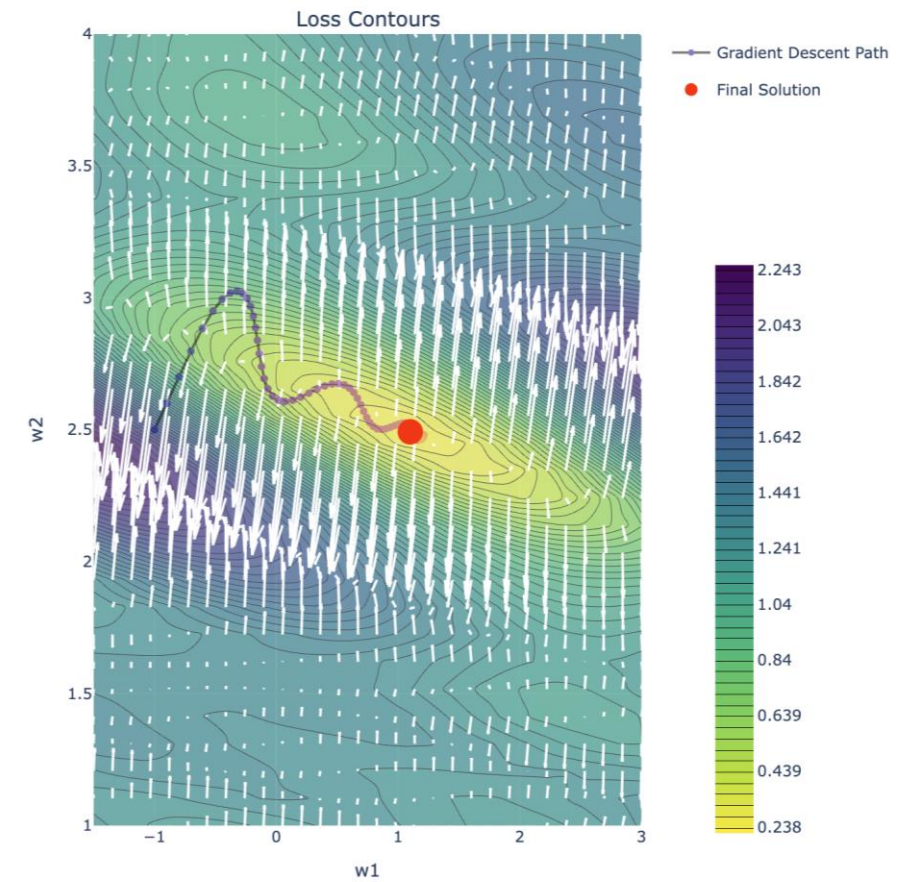
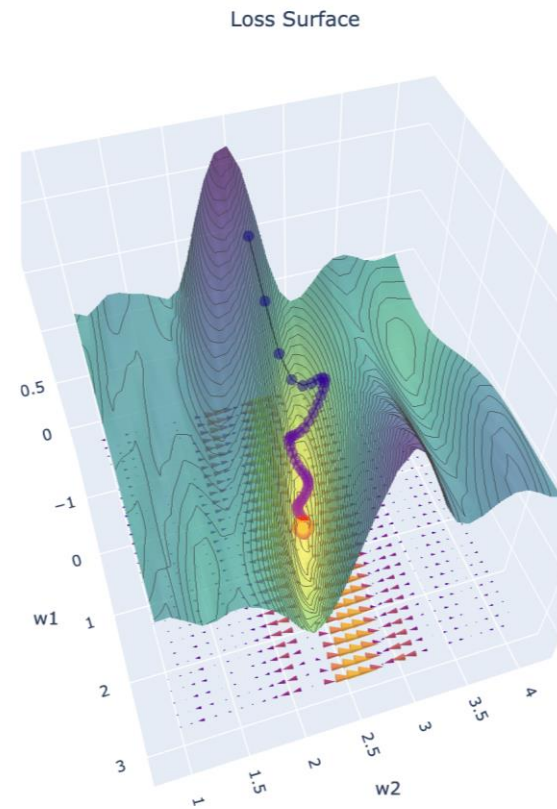
$$\hat{r}_i^{(\tau)} = \frac{r_i^{(\tau)}}{1 - \beta_2^\tau}$$

$$w_i^{(\tau)} = w_i^{(\tau-1)} - \frac{\eta}{\sqrt{\hat{r}_i^{(\tau)} + \delta}} \hat{s}_i^{(\tau)}$$

Addresses issue where early steps are too small.

Demo

Adam



Recap: Batch Gradient Descent

Update the weights by moving in the **opposite direction** of the gradient:

```
 $\mathbf{w}^{(0)} = \text{Initialize}()$   
for  $\tau$  in range(1,  $\tau_{\text{max}}$ ):  
     $\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} + \text{Step}(\nabla J(\mathbf{w}^{(\tau-1)}), \tau, \beta, \eta)$   
    if  $\text{Converged}(\mathbf{w}^{(\tau)}, \mathbf{w}^{(\tau-1)}, \epsilon)$ : terminate
```

$\text{Initialize}()$ typically a small random value (more on this later)

$\text{Converged}(\mathbf{w}^{(\tau)}, \mathbf{w}^{(\tau-1)}, \epsilon)$ stopping condition (e.g., ϵ change $\Delta \mathbf{w}^{(\tau-1)}$)

$\text{Step}(\nabla J(\mathbf{w}^{(\tau-1)}), \tau, \beta, \eta)$ defines how the gradient is used to compute $\Delta \mathbf{w}^{(\tau-1)}$

- This is where Adam would be implemented

The Cost of Batch Gradient Descent

What is the cost of each iteration of batch gradient descent?

```
 $\mathbf{w}^{(0)} = \text{Initialize}()$   
for  $\tau$  in range(1,  $\tau_{\max}$ ):  
     $\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} + \text{Step}(\nabla E(\mathbf{w}^{(\tau-1)}), \tau, \beta, \eta)$   
    if  $\text{Converged}(\mathbf{w}^{(\tau)}, \mathbf{w}^{(\tau-1)}, \epsilon)$ : terminate
```

If we have N training data points in D dimensions:

$$\nabla J(\mathbf{w}^{(\tau-1)}) = \nabla J(\mathbf{w}^{(\tau-1)}; \{(x_n, t_n)\}_{n=1}^N) = \frac{1}{N} \sum_{n=1}^N \nabla J_n(t_n, f_{\mathbf{w}^{(\tau-1)}}(x_n))$$

The cost per update is $\mathcal{O}(ND)$

$\mathcal{O}(N)$

$\mathcal{O}(D)$

The Training Error Estimates the Test Error

As an **optimizer**, our goal has been to minimize the **average training error**:

$$E_{\text{train}}[\mathbf{w}] = \frac{1}{N} \sum_{n=1}^N L(t_n; f_{\mathbf{w}}(x_n))$$

However, we would have preferred to minimize the **expected test error**:

$$E_{\text{test}}[\mathbf{w}] = \mathbb{E}_{p(t_*, x_*)} [L(t_*, f_{\mathbf{w}}(x_*))]$$

- Where $p(t_*, x_*)$ is the unknown distribution over test examples.

If we assume that our training data is drawn from $p(t_*, x_*)$ then for large enough N :

$$E_{\text{train}}[\mathbf{w}] = \frac{1}{N} \sum_{n=1}^N L(t_n; f_{\mathbf{w}}(x_n)) \approx \mathbb{E}_{p(t_*, x_*)} [L(t_*, f_{\mathbf{w}}(x_*))] = E_{\text{test}}[\mathbf{w}]$$

The training error is an empirical estimate of the unknown test error.

Approximating the Gradient

The error (and the gradient) are **empirical expectations** of the **test error**:

$$E(\mathbf{w}^{(\tau-1)}) = \frac{1}{N} \sum_{n=1}^N L(t_n, f_{\mathbf{w}^{(\tau-1)}}(x_n)) \approx \mathbb{E}_{p(t,x)} [L(t, f_{\mathbf{w}^{(\tau-1)}}(x))]$$

The error at a **single point** is *also* a (**noisy**) estimate:

$$E_n(\mathbf{w}^{(\tau-1)}) = L(t_n, f_{\mathbf{w}^{(\tau-1)}}(x_n)) \approx \mathbb{E}_{p(t,x)} [L(t, f_{\mathbf{w}^{(\tau-1)}}(x))]$$

Compute the gradient for a **randomly (stochastically)** sampled **single point**:

$$\underbrace{\nabla E_n(\mathbf{w}^{(\tau-1)})}_{\text{Cheap approx.}} \approx \underbrace{\nabla E(\mathbf{w}^{(\tau-1)}; \{(x_n, t_n)\}_{n=1}^N)}_{\text{Best estimate (but expensive)}} \approx \underbrace{\mathbb{E}_{p(t,x)} [\nabla L(t, f_{\mathbf{w}^{(\tau-1)}}(x))]}_{\text{Ideal but don't know } p(t,x)}$$

Stochastic Gradient Descent

Gradient descent but with the gradient computed on a *single* randomly sampled training data point

```
 $\mathbf{w}^{(0)} = \text{Initialize}()$   
for  $\tau$  in range(1,  $\tau_{\text{max}}$ ):  
     $n = \text{RandomInteger}(0, N)$   
     $\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} + \text{Step}(\nabla E_n(\mathbf{w}^{(\tau-1)}), \tau, \beta, \eta)$   
    if  $\text{Converged}(\mathbf{w}^{(\tau)}, \mathbf{w}^{(\tau-1)}, \epsilon)$ : terminate
```

Gradient of the error on the n^{th} data point.

$$\nabla E_n(\mathbf{w}^{(\tau-1)}) = \nabla L(t_n, f_{\mathbf{w}^{(\tau-1)}}(x_n))$$

- Stochasticity comes from **random gradient estimates** $\nabla E_n(\mathbf{w}^{(\tau-1)})$
- **Cost per step is $\mathcal{O}(D)$** instead of $\mathcal{O}(ND)$ in batch gradient descent

Stochastic Gradient Descent (Shuffling)

In practice, we often **shuffle** and then **iterate** over the entire dataset

```
 $\mathbf{w}^{(0)} = \text{Initialize}();$ 
```

```
 $\tau = 1$ 
```

```
for  $e$  in range(0,  $e_{\text{max}}$ ): # Epochs
```

```
    ShuffleData() # Randomize the Data Ordering
```

```
    for  $n$  in range(0,  $n$ ):
```

```
         $\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} + \text{Step}(\nabla E_n(\mathbf{w}^{(\tau-1)}), \tau, \beta, \eta)$ 
```

```
        if Converged( $\mathbf{w}^{(\tau)}$ ,  $\mathbf{w}^{(\tau-1)}$ ,  $\epsilon$ ): terminate
```

```
     $\tau = \tau + 1$ 
```

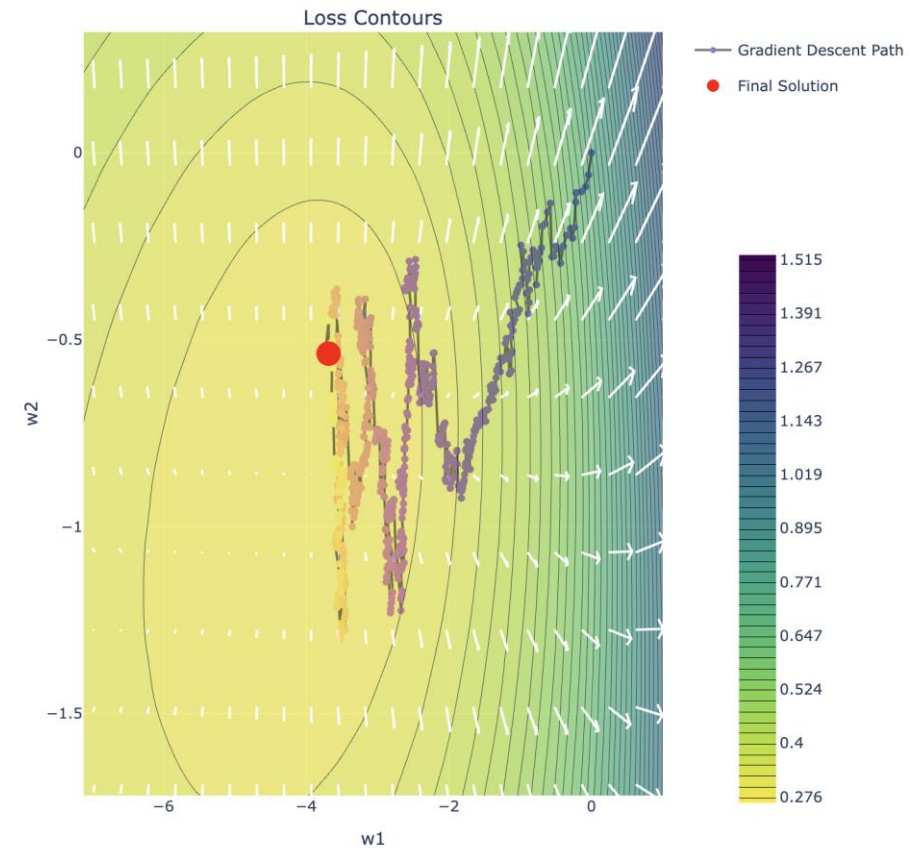
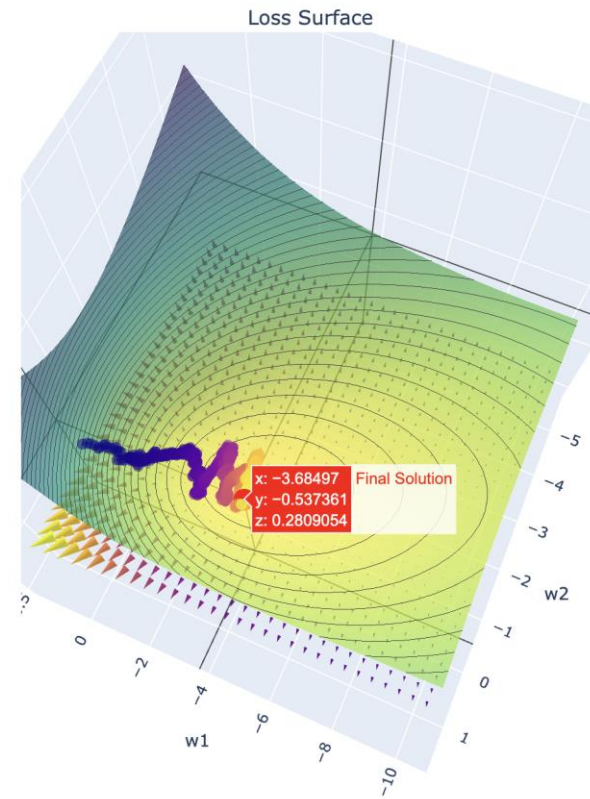
One **epoch** is a complete pass over all the data points.

Often only randomize data once.

Gradient of the error on the n^{th} data point.

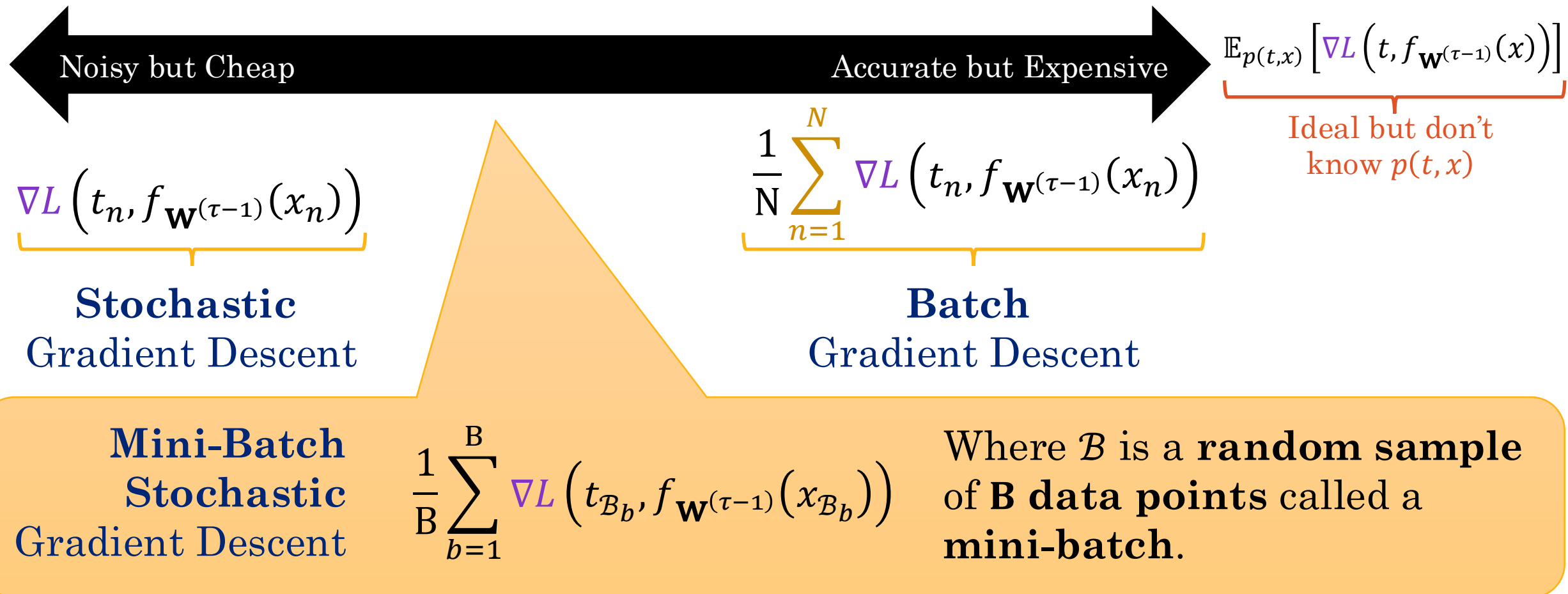
Demo

Stochastic Gradient Descent



A Better Gradient Approximation

Estimating the gradient using a single data point results in **noisy estimates**.



A Spectrum of Stochastic Gradients

We typically refer to any of the sampling based gradient estimation procedures as **Stochastic Gradient Descent (SGD)**

Noisy but Cheap

Accurate but Expensive

$$\underbrace{\nabla L \left(t_n, f_{\mathbf{w}^{(\tau-1)}}(x_n) \right)}$$

**Stochastic
Gradient Descent**

$$\mathcal{O}(D)$$

$$\underbrace{\frac{1}{B} \sum_{b=1}^B \nabla L \left(t_{\mathcal{B}_b}, f_{\mathbf{w}^{(\tau-1)}}(x_{\mathcal{B}_b}) \right)}$$

**Mini-Batch Stochastic
Gradient Descent**

$$\mathcal{O}(BD)$$

$$\underbrace{\frac{1}{N} \sum_{n=1}^N \nabla L \left(t_n, f_{\mathbf{w}^{(\tau-1)}}(x_n) \right)}$$

**Batch
Gradient Descent**

$$\mathcal{O}(ND)$$

Stochastic Gradient Methods

Mini-batch Stochastic Gradient Descent

Gradient descent but with the gradient computed on a **batch of randomly sampled training points**

```
 $\mathbf{w}^{(0)} = \text{Initialize}()$   
for  $\tau$  in range(1,  $\tau_{\text{max}}$ ):  
     $\mathcal{B} = \text{RandBatch}(0, N, \text{size} = B)$   
     $\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} + \text{Step}(\nabla E_{\mathcal{B}}(\mathbf{w}^{(\tau-1)}), \tau, \beta, \eta)$   
    if  $\text{Converged}(\mathbf{w}^{(\tau)}, \mathbf{w}^{(\tau-1)}, \epsilon)$ : terminate
```

$$\begin{aligned}\nabla E_{\mathcal{B}}(\mathbf{w}^{(\tau-1)}) \\ &= \frac{1}{B} \sum_{b=1}^B \nabla L(t_{\mathcal{B}_b}, f_{\mathbf{w}^{(\tau-1)}}(x_{\mathcal{B}_b}))\end{aligned}$$

- **Cost per step is $\mathcal{O}(BD)$** instead of $\mathcal{O}(ND)$ in batch gradient descent
- Note that at batch size $B = 1$ **mini-batch** is the same as **SGD**
 - We often call both algorithms **SGD** (just with a **batch-size hyperparameter**)

Shuffling for Minibatch Gradient Descent

In practice, we often **shuffle** and then **iterate** over the entire dataset

```
 $\mathbf{w}^{(0)} = \text{Initialize}();$ 
```

```
 $\tau = 1$ 
```

```
for  $e$  in range(0,  $e_{\text{max}}$ ): # Epochs
```

```
    ShuffleData() # Randomize the Data Ordering
```

```
    for  $n$  in range(0,  $n$ , step= $B$ ):
```

```
         $\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} + \text{Step}(\nabla E_{n, \dots, n+B}(\mathbf{w}^{(\tau-1)}), \tau, \beta, \eta)$ 
```

```
        if Converged( $\mathbf{w}^{(\tau)}, \mathbf{w}^{(\tau-1)}, \epsilon$ ): terminate
```

```
         $\tau = \tau + 1$ 
```

One **epoch** is a complete pass over all the data points.

Often only randomize data once.

Gradient of the error on the next B data points.

How do I Pick the Batch Size?

Larger batch size:

- **Better gradient estimate**
- **Enables hardware parallelism** – can compute gradient for each point in parallel

Smaller batch size:

- **Cheap to compute** → take more steps
- **Greater stochasticity** can help with local minima

Typical design decision:

- Increase batch size to **saturate hardware parallelism**
- **Increase learning rate** proportional to batch size (take bigger steps)

Demo

Mini-batch Gradient Descent

